

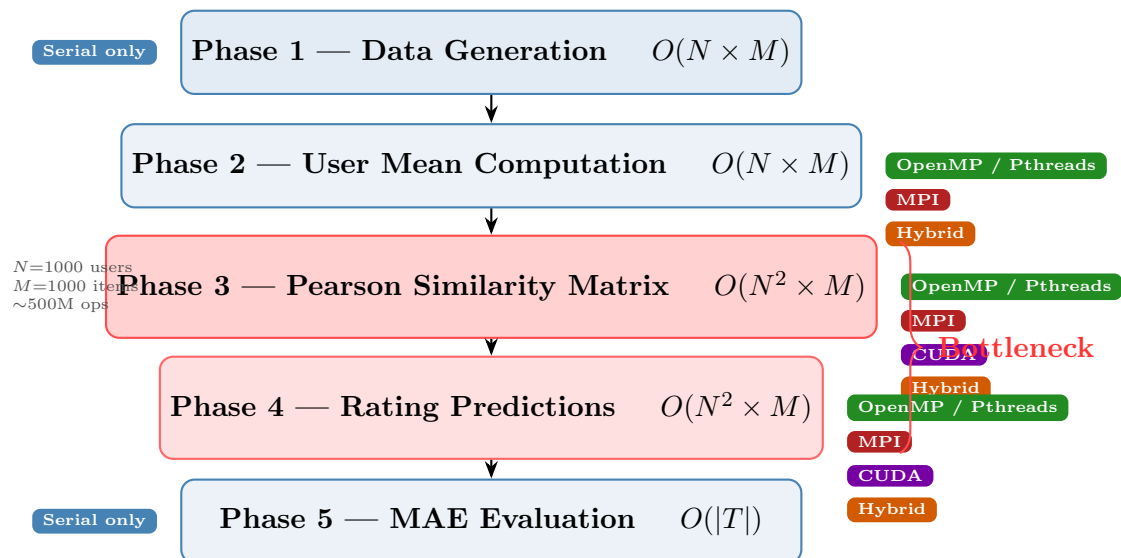
Parallel Programming Concepts Applied to the Pearson Correlation Recommender System

EC7207/EE7218 High Performance Computing • Group 11

This section presents diagrams and descriptions that illustrate how each of the five parallel programming paradigms — **OpenMP**, **POSIX Threads (Pthreads)**, **MPI**, **CUDA**, and **Hybrid MPI+OpenMP** — was applied to accelerate a User-Based Collaborative Filtering recommender system. The system computes Pearson Correlation similarity between 1 000 users over 1 000 items, predicts unseen ratings, and evaluates Mean Absolute Error (MAE).

System Overview: The 5-Phase Algorithm Pipeline

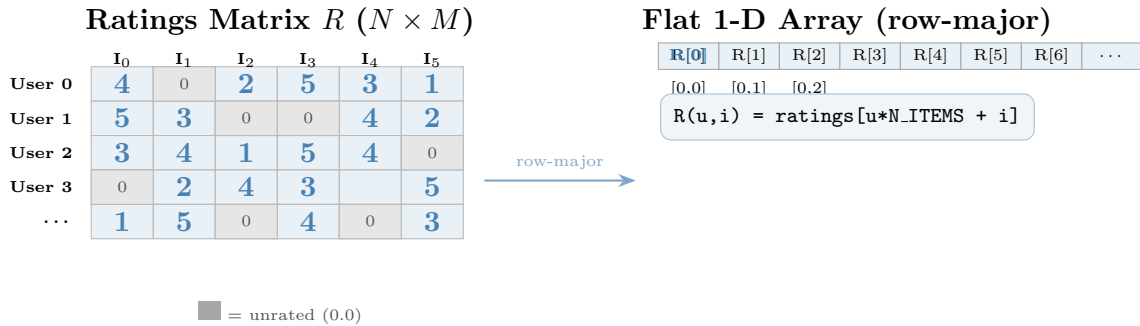
Diagram 1 — Algorithm Flow and Parallelisation Targets



Description. The recommender system executes in five sequential phases. Phases 1 and 5 are inherently serial: data generation uses a fixed random seed (**SEED=42**) to ensure identical inputs across all versions, and MAE evaluation is fast ($O(|T|)$) and not a bottleneck. Phases 2–4 are parallelised. The dominant cost lies in Phases 3 and 4, each of $O(N^2 \times M)$ complexity — approximately 500 million floating-point operations for a 1000×1000 matrix. These are the targets for acceleration with OpenMP, Pthreads, MPI, CUDA, and Hybrid parallelism.

Data Structure: Ratings Matrix and Row Partitioning

Diagram 2 — Flat Row-Major Storage and Work Division

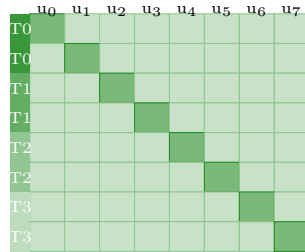
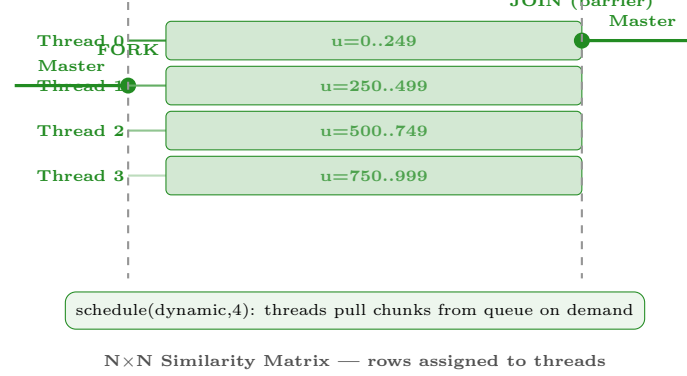


Description. The ratings matrix R ($N_{\text{users}} \times M_{\text{items}}$) is stored as a flat 1-D array in row-major order. Entry $R[u, i]$ is accessed via the macro $R(u,i) = \text{ratings}[u \times N_ITEMS + i]$. A value of 0.0 indicates an unrated cell. This storage layout enables a single `calloc` call per matrix, maximises CPU cache utilisation during row scans, and simplifies inter-rank communication in MPI (`MPI_Allgatherv` sends contiguous row blocks). The same layout is used for the $N \times N$ similarity matrix (`sim_matrix`) and the predictions matrix.

OpenMP — Shared-Memory Fork-Join Parallelism

Diagram 3 — OpenMP Applied to the Similarity Phase

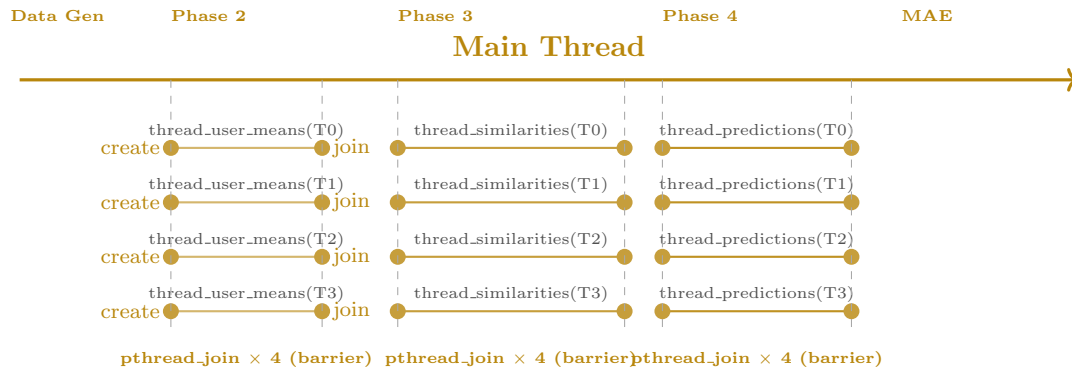
Parallel Region: Phase 3 (Similarity) & Phase 4 (Predictions)



Description. OpenMP parallelises Phases 2–5 using compiler directives (`#pragma omp`). In Phase 3 (similarity), each thread receives a dynamic chunk of outer-loop iterations u and computes Pearson correlations for all $v > u$ in that chunk, writing to disjoint rows of `sim_matrix[]`. `schedule(dynamic,4)` is used because the inner loop shrinks as u increases — thread 0 processes far more pairs than thread 3, so static equal-chunk assignment would cause severe load imbalance; dynamic scheduling lets idle threads pull new chunks from a shared queue. In Phase 4 (predictions), each thread allocates its own private neighbour buffer (`malloc` inside `#pragma omp parallel`) to avoid any shared-state race. An implicit OpenMP barrier at the end of each parallel region ensures phase ordering. The serial fraction (Phase 1 data generation, Phase 5 MAE output) is $\approx 2\text{--}3\%$, limiting maximum speedup to $\approx 35\text{--}50\times$ by Amdahl's Law.

Pthreads — Manual POSIX Thread Management

Diagram 4 — Pthreads Lifecycle and Work Partitioning



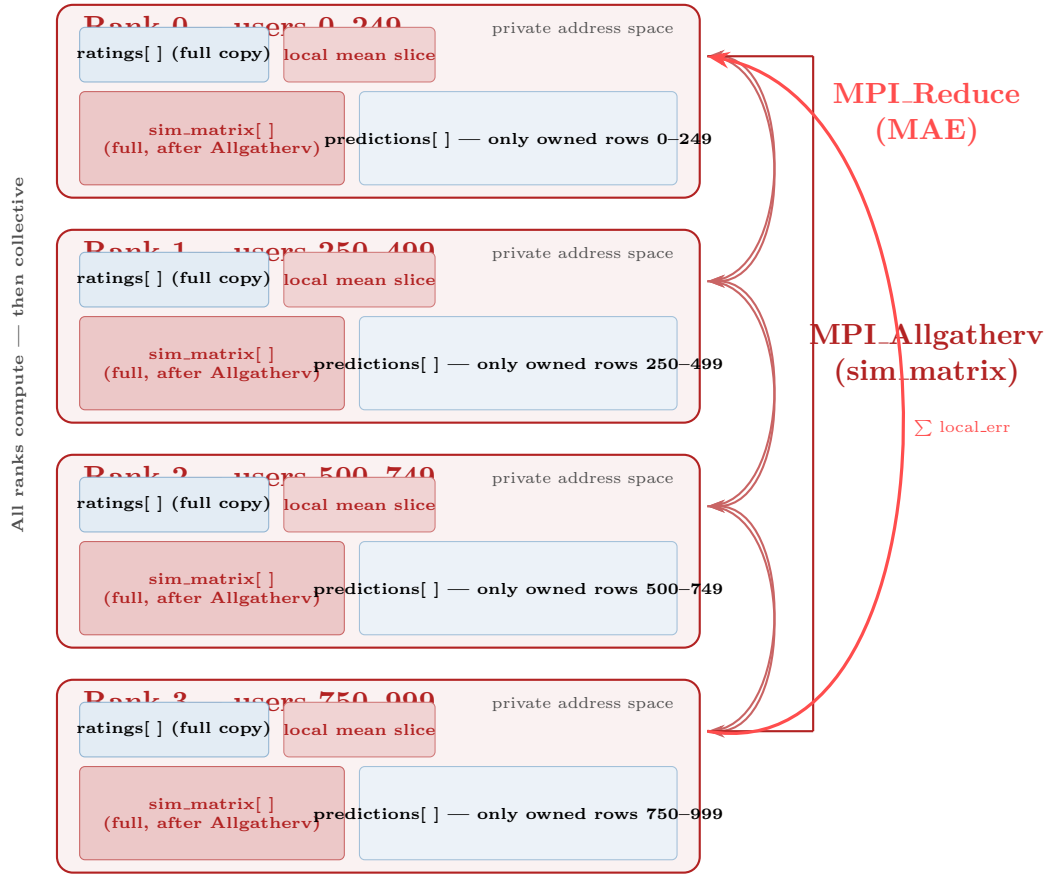
Work Range Assignment (ceiling division)



Description. The Pthreads version explicitly creates and joins threads for each of the three parallel phases using a helper function `run_parallel()`. A `ThreadArgs` struct `{tid, nthreads}` is passed to each thread, from which the thread independently computes its row range via ceiling division: $\text{chunk} = (\text{N_USERS} + \text{nthreads} - 1) / \text{nthreads}$. Thread t owns rows $[t \times \text{chunk}, (t + 1) \times \text{chunk})$. Since all threads write to disjoint index ranges of `sim_matrix[]` and `predictions[]`, no mutex is required. The private neighbour buffer `nbrs[]` is allocated inside each thread function — each thread gets its own heap allocation via `malloc()`, which is thread-safe. `pthread_join` after the last thread serves as a phase barrier, guaranteeing that all similarity values are written before the prediction phase reads them.

MPI — Distributed-Memory Row Partitioning

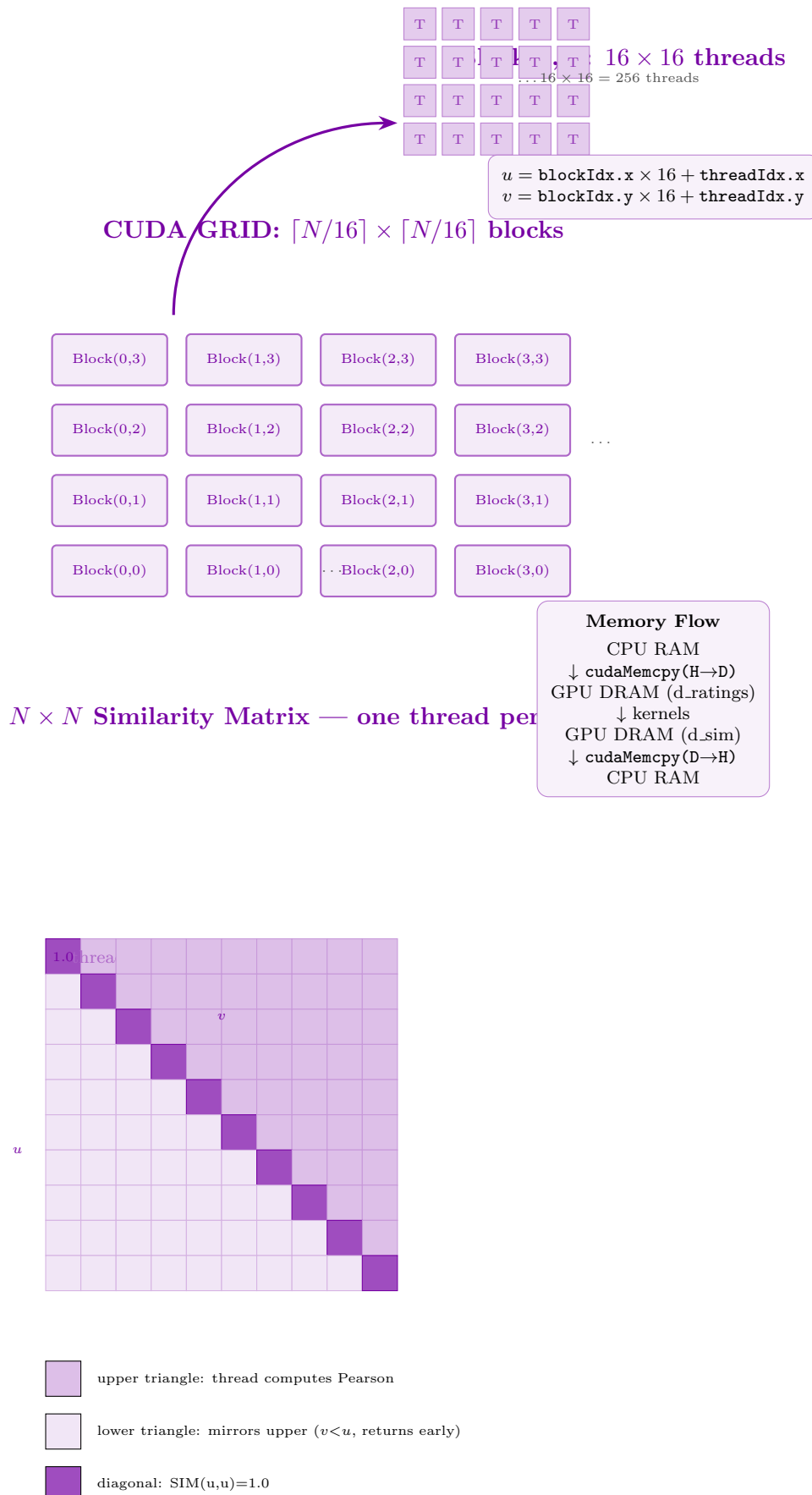
Diagram 5 — MPI Process Layout and Collective Communication



Description. `mpirun -np P` launches P independent process copies, each with a private address space. Rows are assigned via ceiling division: rank r owns users $[r \cdot \lceil N/P \rceil, (r+1) \cdot \lceil N/P \rceil)$. All ranks independently reproduce the full ratings matrix using the same fixed seed (`srand(42)`), eliminating the need to broadcast it. After Phase 2, each rank holds its mean slice; `MPI_Allgatherv` assembles the complete `user_mean[]` on every rank. Similarly, after Phase 3 each rank sends its row block of `sim_matrix` and receives the full $N \times N$ matrix via `MPI_Allgatherv`, enabling Phase 4 predictions with no further communication. Phase 5 MAE is computed locally per rank and summed to rank 0 with `MPI_Reduce`. `MPI_Barrier` brackets each timed section to capture true wall-clock cost including load imbalance.

CUDA — GPU Massively Parallel Computation

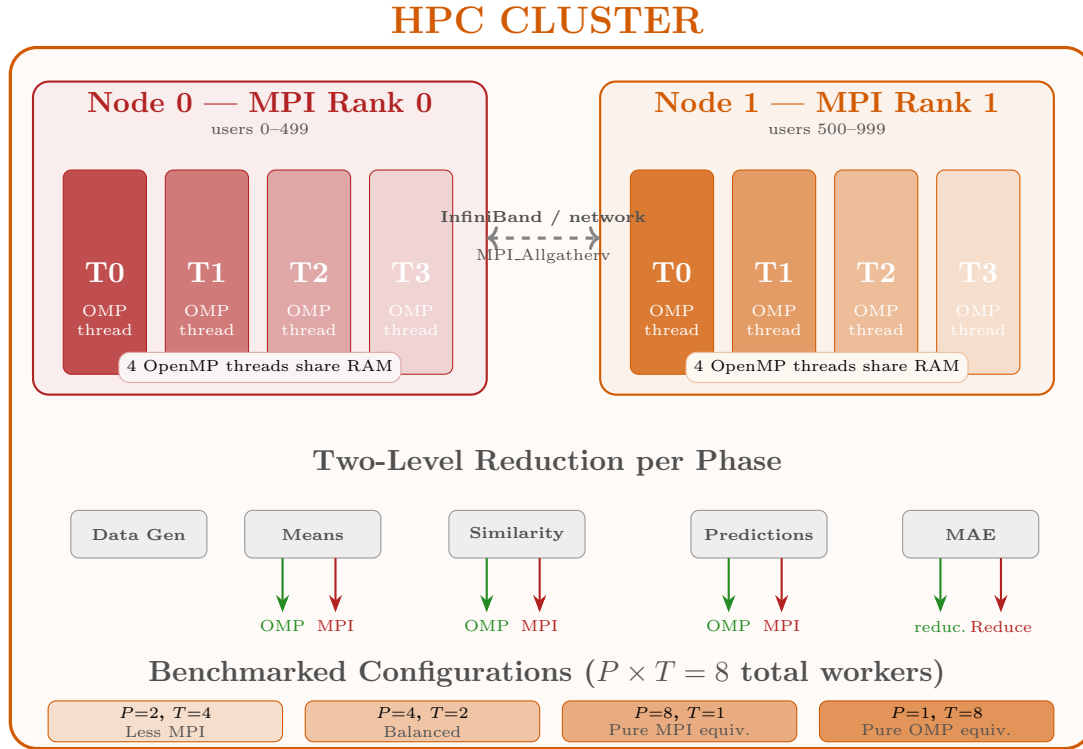
Diagram 6 — CUDA Thread Hierarchy Mapped to Similarity Matrix



Description. The CUDA version maps each (u, v) pair of the similarity matrix to one GPU thread. A 2-D grid of $\lceil N/16 \rceil \times \lceil N/16 \rceil$ blocks, each containing $16 \times 16 = 256$ threads, is launched for `kernel_similarity`. Thread (u, v) computes Pearson correlation for that pair and writes both `SIM(u,v)` and `SIM(v,u)` atomically (race-free because each (u, v) pair is unique to one thread). Threads in the lower triangle ($v < u$) return immediately to avoid redundant computation. The prediction kernel (`kernel_predictions`) uses a 2-D grid over (u, item) pairs and maintains an in-register min-heap of size `TOP_K=20` per thread (since `qsort` is unavailable in device code). Kernel execution time is measured precisely with CUDA events (not CPU wall-clock timers, because kernel launches are asynchronous). Data is transferred between CPU RAM and GPU DRAM via `cudaMemcpy` over the PCIe bus before and after computation.

Hybrid MPI+OpenMP — Two-Level Parallelism

Diagram 7 — Hybrid Architecture on a Multi-Node Cluster



Description. The Hybrid version combines MPI and OpenMP to exploit both levels of parallelism present in a real HPC cluster. At the **coarse level**, MPI partitions N_{users} rows across P ranks (processes), each running on a separate node with its own private RAM. At the **fine level**, within each MPI rank, T OpenMP threads cooperate over the rank's row slice using shared memory, with no additional MPI messages. Total effective workers = $P \times T$, but MPI collectives (MPI_Allgatherv, MPI_Reduce) involve only P messages — far fewer than a pure MPI program with $P \times T$ processes. MPI_Init_thread with MPI_THREAD_FUNNELED is used because MPI calls occur only from the main thread (outside #pragma omp parallel regions). Phase 5 uses a two-level reduction: OpenMP's reduction(+:local_err) merges per-thread partial sums within each rank, then MPI_Reduce sums per-rank totals to rank 0. Four configurations (2×4, 4×2, 8×1, 1×8) were benchmarked, all with 8 total workers, to reveal how different MPI-to-OpenMP ratios affect performance.

Summary: Parallel Concept Application Table

Version	Memory Model	Work Division	Synchronisation	Key API / Directive
Serial	Shared (1 core)	None	None	<code>clock_gettime</code>
OpenMP	Shared (all threads)	<code>schedule(dynamic, outer loop u)</code>	Implicit barrier at }	<code>#pragma omp parallel for, reduction</code>
Pthreads	Shared (all threads)	Ceiling-division <code>work_range()</code>	<code>pthread_join</code> loop	<code>pthread_create, pthread_join</code>
MPI	Distributed (private per rank)	Row blocks via <code>get_range()</code>	<code>MPI_Barrier</code> + collective ops	<code>MPI_Allgatherv, MPI_Reduce</code>
CUDA	Host + GPU DRAM	1 thread per (u, v) or (u, i) cell	<code>cudaDeviceSynchronize</code>	<code>global__ kernels, dim3 launch</code>
Hybrid	Distributed (MPI) + Shared (OMP)	MPI rows, OMP within rank	OMP barrier then MPI collective	<code>MPI_Init_thread, #pragma omp</code>

Overall Parallelisation Strategy. The bottleneck phases (Similarity, $O(N^2M)$, and Predictions, $O(N^2M)$) account for $> 97\%$ of execution time. All five parallel versions target these phases with the same algorithmic approach — row-level work distribution — but differ in how they implement it: shared memory (OpenMP, Pthreads) vs. distributed memory (MPI) vs. GPU threads (CUDA) vs. a combination (Hybrid). The serial fraction (fixed-seed data generation) is kept in all versions to ensure numerical reproducibility: every version produces the same `sim_matrix` checksum and MAE (within ± 0.001), confirming correctness. Speedup is measured as $S(p) = T_{\text{serial}}/T_{\text{parallel}}(p)$ and efficiency as $E(p) = S(p)/p$, benchmarked across thread/process counts of 1, 2, 4, and 8.